

Я добавляю две сущности, которых не хватало в ДЗ №1 для полноценной логики управления проектами:

task_dependency - зависимости между задачами (задача-предшественник - > зависимая задача). Это классическая связь M:N «задача <-> задача» (самореферентная), которую нельзя выразить одним внешним ключом.

attachment - вложения (файлы) к задачам (опционально - к конкретному комментарию).

Обе таблицы поддерживают частые запросы из ДЗ №1 (например, «показать блокирующие задачу зависимости» и «вывести вложения задачи»), а также добавляют бизнес-ограничения.

№	Ограничение	Выполняемый запрос (SQL)	Сообщение СУБД (SQLERRM)	Понятное сообщение для пользователя	Как исправить
1	CHECK	INSERT INTO attachment (... , file_size) VALUES (... , -5, ...)	new row for relation "attachment" violates check constraint "chk_attachment_size"	«Размер файла должен быть положительным числом»	Указать корректное значение file_size > 0
2	FOREIGN KEY	INSERT INTO task_dependency (... , depends_on_task_id) VALUES (... , 999999)	insert or update on table "task_dependency" violates foreign key constraint "fk_dependency_depends_on"	«Нельзя создать зависимость от несуществующей задачи»	Указать task_id существующей задачи
3	UNIQUE	INSERT INTO employee (... , email) VALUES (... , 'ivan@example.com')	duplicate key value violates unique constraint "employee_email_key"	«Сотрудник с таким email уже зарегистрирован»	Использовать другой уникальный email
4	NOT NULL	INSERT INTO task (project_id, status_id, priority) VALUES (...) (без title)	null value in column "title" of relation "task" violates not-null constraint	«Задача должна иметь название»	Указать обязательное поле title
5	TRIGGER	INSERT INTO task_dependency (task_id, depends_on_task_id) VALUES (v_b, v_a) после существующей связи (v_a, v_b)	circular task dependency detected	circular task dependency detected	Пересмотреть связи зависимостей и удалить лишнюю

Краткие выводы

М:N реализовано таблицей-связкой (task_dependency) с составным РК - это единственный нормализованный способ сохранить ссылочную целостность и возможность эффективных запросов в обе стороны.

Ограничения спроектированы под частые запросы ДЗ №1:

- индекс (status_id, deadline) - ускоряет «просроченные задачи»;
- assignment(employee_id) + task(status_id) - для «сотрудников с количеством активных задач»;
- task_status_history(task_id) - для «истории смены статуса»;
- составной РК в task_dependency + индекс по depends_on_task_id - для обхода зависимостей в обе стороны;
- ни один типовой запрос не требует полного сканирования таблицы.

Бизнес-логика закреплена на уровне БД, а не только приложения. запрет само-зависимости (CHECK), запрет отрицательного размера файла (CHECK), уникальность email и пути файла (UNIQUE), невозможность «пустого» названия задачи (NOT NULL), запрет циклических зависимостей (TRIGGER) и запрет бессмысленного «перехода в тот же статус» (CHECK на task_status_history).

DROP ... CASCADE != ON DELETE CASCADE: первый ломает схему (снимает FK с дочерних таблиц, оставляя данные «осиротевшими»), второй штатно удаляет связанные строки, сохраняя целостность.